

IC3/PDR

A.R. Bradely

Presented by Lucky M. Kispotta

`luckymk.mcs2024@cmi.ac.in`

Chennai Mathematical Institute

2025-12-05

Table of Contents

Introduction

IC3/PDR

Key Ideas

Implementation details

Content

Introduction

IC3/PDR

Key Ideas

Implementation details

Preliminaries

Verification

Given a program, we need to check if the program is correct. This is done via *SAT* checks.

For example,

function *nodividebyzero*(x, y):

```
1  | ....  
2  | assert( $y \neq 0$ )  
3  |  $z = \frac{x}{y}$   
4  | .....
```

A Property which should hold every time it reached location 2 and 3 is that $y \neq 0$.

Transition System

- A program can be thought of as a Transition System $TS = (S, \text{init}, T, P)$.
- A state is a valuation of program variables.
- The steps in a program are converted to a *Transition System*.
- A transition relation T is a formula which satisfies if there exists a transition from state s to s' in the program.
- Example,

$T(s, s') = \top$ when there is a step in the program from $s \rightarrow s'$.

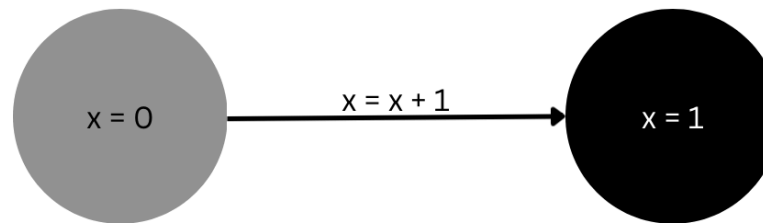


Figure 1: Transition from $s \rightarrow s'$

CTI and Inductive Invariant Property

Property

A property is a logical formula on the variables of the program state.

- The given property P is safe *iff* $\exists$ an inductive invariant Inv .
- Initiation : $\text{Init}(s) \Rightarrow \text{Inv}(s)$
- Inductive: $\text{Inv}(s) \wedge T(s, s') \wedge \text{Inv}(s')$
- Consequence: $\text{Inv}(s) \Rightarrow P(s)$

CTI and Inductive Invariant Property



Figure 2: CTI and inductive invariant P^*

CTI

Any reachable state s which goes to a bad state s' in one or more transitions is called “Counter-example to induction” (CTI).

Some Model Checking Techniques

Some model checking techniques which helps in validating the safety of P given a transition system are :

- *k - induction*

The drawbacks are that this method is not scalable for large k .
It unrolls the transition system in each SAT check.

$$P(s_1) \wedge T(s_1, s_2) \wedge P(s_2) \dots T(s_{k-1}, s_k) \Rightarrow P(s_k)$$

- *Fixpoint Model Checking*

The drawbacks are that this method requires computing the *pre-image* and *image*.

Does not scale due to quantification elimination.

Content

Introduction

IC3/PDR

Key Ideas

Implementation details

Motivation

- The *IC3/PDR* algorithm is one of the most efficient algorithms which solves this problem of verifying the safety of a Property w.r.t Transition system.
- Use only one length transition enrolling per SAT call.
- Strengthen using learned clauses.
- The goal of this algorithm is to return a inductive invariant if the property P is **SAFE** otherwise return **FALSE**.

[*References : SAT-Based Model Checking Without Unrolling by Aaron Bradely*]

Inputs/Outputs

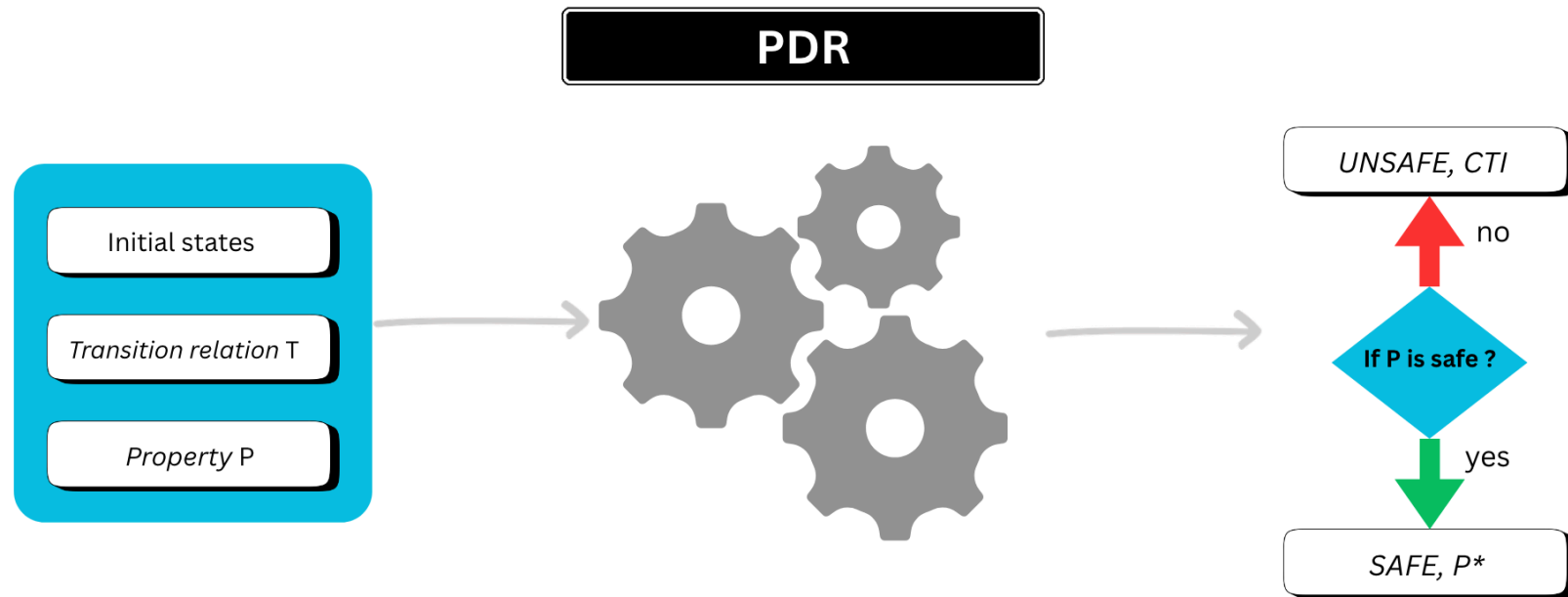


Figure 4: PDR input/outputs

Content

Introduction

IC3/PDR

Key Ideas

Implementation details

Frames and Algorithmic Invariants

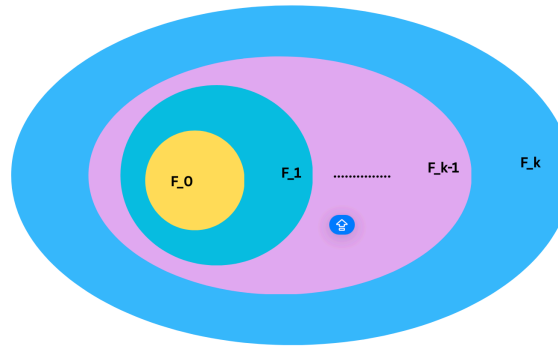


Figure 5: Visualization of frames

The algorithm maintains a set of candidate inductive invariants (Frames). PDR is an iterative algorithm that preserves the following invariants:

Algo Inv 1: Frame F_0 contains the set of initial states.

Algo Inv 2: The series of frames is monotone, $F_i \subseteq F_{i+1}$.

Algo Inv 3: None of the frames contain $\neg p$.

Algo Inv 4: Every F_{i+1} is an over-approximation of 1-step image of states in F_i .

Algo Inv 5: Clauses of F_{i+1} is a subset of clauses of F_i .

Extending Frame

- At Frame i , the algorithm tries to extend the set of frames to $i + 1$.
- It initializes frame $i + 1$ with P .

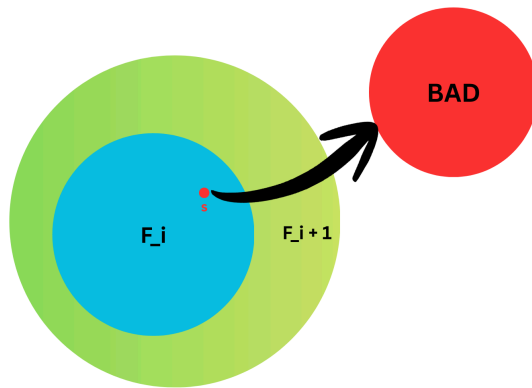


Figure 6: s is the *CTI*

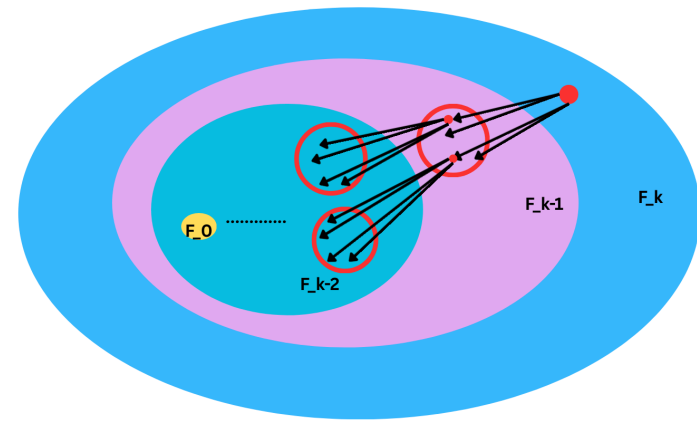


Figure 7: Recursively block CTIs in the previous frames

- The algorithm blocks the *CTI* at frame i and tries recursively blocking all the predecessor of s in Frame $k - 1$.
- If some predecessor $\in \text{Init}$ set then this is a concrete counterexample. And this is a counterexample trace.

Generating CTI[k] while extending frame

Checking if a CTI exists at frame k :

$$\text{SAT}[s \wedge T(s, s') \wedge \neg P(s') \wedge F_k(s)]$$

Check performed to find the predecessor:

$$\text{SAT}[T(s, s') \wedge F_{i-1}(s)]$$

s' is the CTI observed at frame k .

An improvement over this is adding $\neg s$. This forbids CTIs having self loops from recurring.

$$\text{SAT}[T(s, s') \wedge F_{i-1}(s) \wedge \neg s]$$

Blocking

- Given a CTI s' at Frame i , it is blocked at frame j , $1 \leq j < i$. That is,
$$\text{UNSAT}[F_j(s) \wedge T(s, s') \wedge \neg s]$$
- If a CTI reaches F_0 before being blocked the CEX $\rightarrow \text{ABORT Unsafe}$.
- Blocking is done by adding $\neg s$ to Frame $1 \rightarrow i$. This is called **STRENGTHENING**.

Faster convergence using Propagation Phase

Soundness Lemma

If a clause c is *relatively inductive* at frame i then it is sound to add it in all frames $> i$.

- Another optimization used in PDR is :

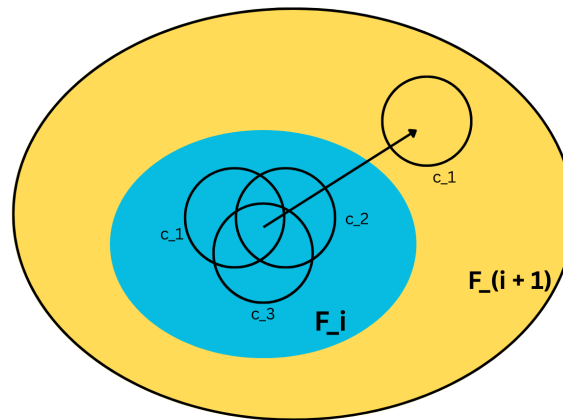


Figure 12: clause $c \in \text{Frame } i$ holds for all states in Frame $i + 1$

The VALIDITY check for this is :

$$\text{VALID}[c(s) \wedge c(s') \wedge T(s, s') \wedge F_i(s)]$$

Overview of the algorithm

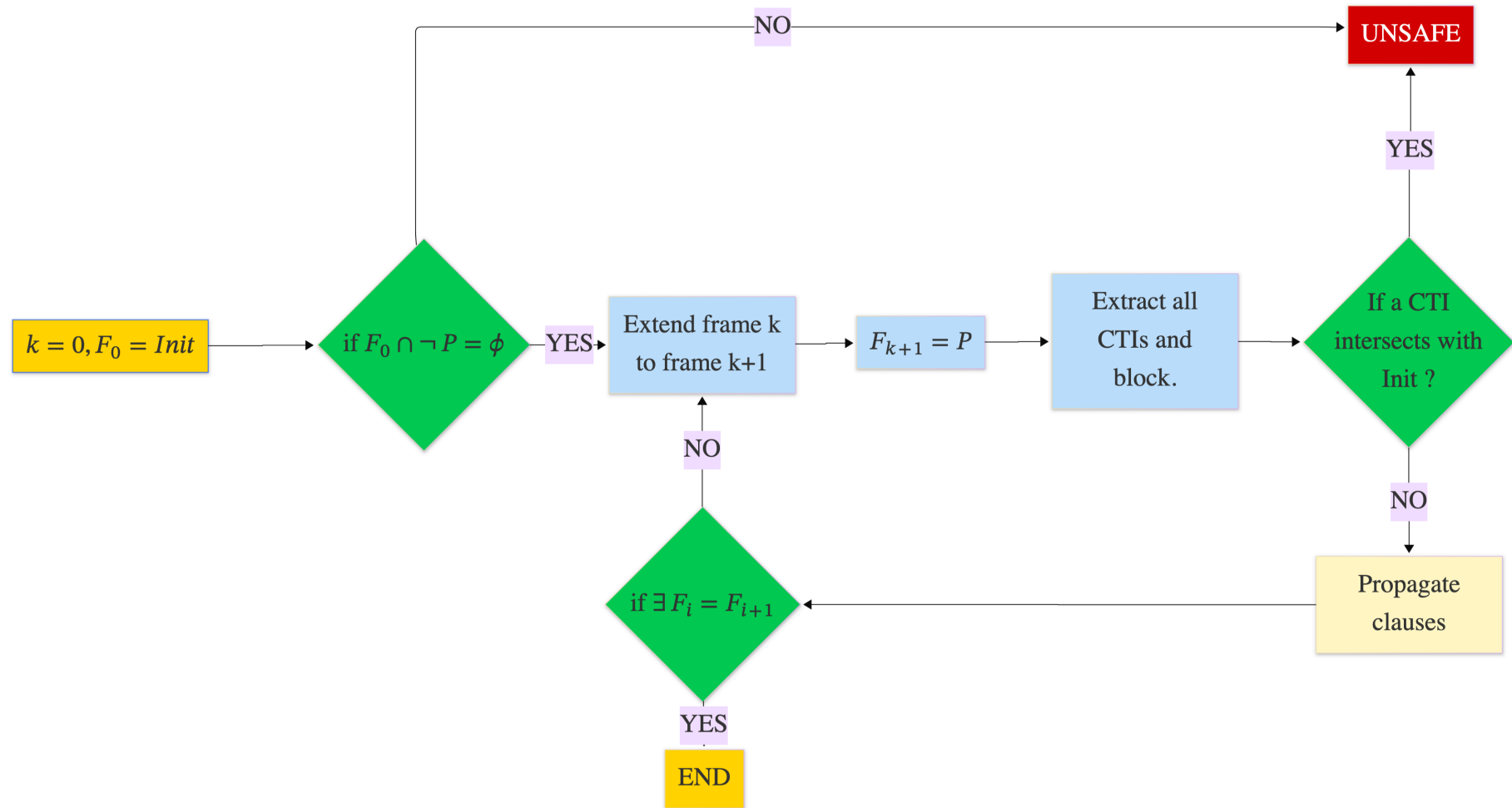


Figure 14: PDR flowchart

Content

[Introduction](#)

[IC3/PDR](#)

[Key Ideas](#)

[Implementation details](#)

PDR Pseudocode

PDR(Init, T , P):

```
1  if SAT[Init  $\wedge \neg P$ ] :  
2    | return False  
3   $F_0 = \text{Init}$   
4   $k = 0$   
5  while True:  
6    | extendFrontier(Init,  $T$ ,  $P$ ,  $k$ )  
7    | if  $k > 0$ :  
8      | propagateClause( $T$ ,  $P$ ,  $k$ )  
9    | if  $\exists F_i == F_{i+1}$ :  
10   | | return  $F_i$   
11   |  $k = k + 1$ 
```

Extend frontier pseudocode

ExtendFrontier(Init, T , P , k):

```
1   $F_{k+1} = P$ 
2  while SAT[ $s \wedge F_k(s) \wedge T(s, s') \wedge \neg P(s')$ ]:
3  |    $RemoveCTI(Init, T, P, k, s)$ 
```

Adds frame $k + 1$ as P and strengthens frame k .

RemoveCTI pseudocode

RemoveCTI(Init, T , P , k , s):

```
1  if SAT[Init  $\wedge$   $s$ ] :  
2    | return False  
3    Block  $s$  in frames  $F_1 \dots F_k$   
4    while SAT[ $F_{k-1}(s') \wedge T(s', s) \wedge \neg s'$ ]:  
5    | RemoveCTI(Init,  $T$ ,  $P$ ,  $k - 1$ ,  $s'$ )
```

Blocks CTI s at frame k and recursively blocks its predecessors (which are also CTIs).

Propagate clauses

PropagateClauses(k):

```
1  for  $i \in [1, k]$ :  
2      for  $c \in \text{clause}(F_i)$ :  
3          if  $\text{VALID}[F_i(s) \wedge T(s, s') \wedge c(s')]$ :  
4               $F_{i+1} = F_{i+1} \wedge c$ 
```

Propagates clauses from Frame k to Frame $k + 1$.

Implementation details/discussions

- The algorithm has been implemented in Python.
- Z3 has been used as the SAT solver.
- Optimization of syntactically checking if a block is already blocked in a frame has been implemented.
- The optimization of using a priority queue instead of recursive function calls in *RemoveCTI* has not been implemented.

Thanks for Listening.

Bye Bye

